

Partie III : Calcul scientifique

Calcul de la matrice inverse d'une matrice carrée inversible

Méthode : Élimination de 'Gauss-Jordan'

En mathématiques, l'élimination de « Gauss-Jordan », aussi appelée pivot de Gauss, nommée en hommage à Carl Friedrich Gauss et Wilhelm Jordan, est un algorithme de l'algèbre linéaire pour déterminer les solutions d'un système d'équations linéaires, pour calculer l'inverse d'une matrice carrée inversible ou pour déterminer le rang d'une matrice...

Pour calculer la matrice inverse d'une matrice carrée M inversible, la méthode de l'élimination de « Gauss-Jordan » consiste à effectuer des transformations, simultanément sur les deux matrices : la matrice M, et la matrice identité E de même dimension que M.

Le but de la méthode de l'élimination de « Gauss-Jordan » est de transformer la matrice M en matrice identité, tout en effectuant simultanément les mêmes opérations (effectuer les mêmes échanges, utiliser les mêmes valeurs des pivots ...), sur les deux matrices M et E. La matrice E contient, alors, la matrice inverse de la matrice M initiale.

Exemple :

Matrice carrée inversible M :	Matrice identité E, de même dimension que M :
$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$	$\begin{bmatrix} 1. & 0. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 0. & 1. & 0. \\ 0. & 0. & 0. & 1. \end{bmatrix}$
Après les transformations, les deux matrices M et E deviennent ainsi :	E contient l'inverse de la matrice M initiale :
$\begin{bmatrix} 1. & 0. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 0. & 1. & 0. \\ 0. & 0. & 0. & 1. \end{bmatrix}$	$\begin{bmatrix} 0.8 & -0.1 & 0. & -0.5 \\ -1. & 0.5 & 0. & 0.5 \\ 5. & -2. & -1. & -3. \\ 5.4 & -2.3 & -1. & -3.5 \end{bmatrix}$

On suppose que le module **numpy** est importé :

import numpy as np

Le module **numpy** contient les méthodes et les fonctions permettant de manipuler les matrices.

On rappelle que les indices des listes et tableaux en Python commencent à 0.

Exemple :

M = np.array([[1, -1, 2, -2], [3, 2, 1, -1], [2, -3, -2, 1], [-1, -2, 3, -3]], float)

Cette instruction permet de créer la matrice **M** suivante :

1.	-1.	2.	-2.
3.	2.	1.	-1.
2.	-3.	-2.	1.
-1.	-2.	3.	-3.

III. 1- Matrice inversible :

Une matrice carrée est inversible, si son déterminant est différent de 0.

Le module **numpy** contient un sous-module appelé : **linalg**. Ce dernier contient la méthode : **det()** qui reçoit en paramètre une matrice, et qui retourne la valeur du déterminant de cette matrice.

Écrire la fonction : **inversible(M)**, qui reçoit en paramètre une matrice carrée **M**, et qui retourne **True** si la matrice **M** est inversible, sinon, elle retourne **False**.

III. 2- Matrice identité :

Écrire la fonction : **identite(n)**, qui reçoit en paramètre un entier strictement positif **n**, et qui crée et retourne la matrice identité d'ordre **n** (**n** lignes et **n** colonnes), de nombres réels.

Exemple :

La fonction **identite(4)** retourne la matrice identité **E** d'ordre 4 suivante :

1.	0.	0.	0.
0.	1.	0.	0.
0.	0.	1.	0.
0.	0.	0.	1.

III. 3- Opération de transvection :

Écrire une fonction : **transvection(M, p, q, r)**, qui reçoit en paramètres une matrice **M**, deux entiers **p** et **q** représentant les indices de deux lignes dans la matrice **M**, et un nombre réel **r**. Dans la matrice **M**, la fonction remplace la ligne **p** par la ligne résultat de la combinaison linéaire des deux lignes **p** et **q**, suivante : **Mp + r * Mq**.

Exemple :

Après l'appel : **transvection(M, 2, 0, -3.)**, la matrice **M** devient :

1.	-1.	2.	-2.
3.	2.	1.	-1.
2.	-3.	-2.	1.
-1.	-2.	3.	-3.

1.	-1.	2.	-2.
-1.	0.	-8.	7.
-1.	-2.	3.	-3.

III. 4- Échanger deux lignes dans une matrice :

Écrire la fonction : **echange_lignes(M, p, q)**, qui reçoit en paramètres une matrice **M**, et deux entiers **p** et **q** représentant respectivement deux indices de deux lignes dans la matrice **M**. la fonction échange les lignes **p** et **q** dans la matrice **M**.

Exemple :

Après l'appel : **echange_lignes(M, 1, 3)**, les lignes 1 et 3 de la matrice **M** sont échangées :

1.	-1.	2.	-2.
3.	2.	1.	-1.
2.	-3.	-2.	1.
-1.	-2.	3.	-3.

1.	-1.	2.	-2.
-1.	-2.	3.	-3.
2.	-3.	-2.	1.
3.	2.	1.	-1.

III. 5- Recherche de la ligne du pivot dans une colonne de la matrice :

Écrire la fonction : `ligne_pivot (M, c)` , qui reçoit en paramètres une matrice **M**, et un entier **c** représentant une colonne dans la matrice **M**.

La fonction retourne l'indice **p** tel que : $|M[p, c]| = \max \{ |M[i, c]| \mid 0 \leq i < c \}$

Exemple :

Si la matrice **M** est la suivante :

-1.	1.	5.	2.
10.	1.	-6.	-4.
8.	6.	-3.	7.
-9.	5.	7.	-2.

La fonction `ligne_pivot (M, 2)` retourne l'indice : 1, car, dans la colonne 2, l'élément -6.0 est le plus grand en valeur absolue, parmi les éléments des lignes 0, 1 et 2.

III. 6- Transformer la matrice M en matrice triangulaire inférieure :

Écrire la fonction : `triangulaire_inf (M, E)` , qui reçoit en paramètre une matrice carrée inversible **M**, et une matrice identité **E** de même dimension que **M**. La fonction transforme la matrice **M** en matrice triangulaire inférieure. Les transformations effectuées sur la matrice **M**, doivent être effectuées simultanément sur la matrice **E**.

Exemple :

Après l'appel de la fonction `triangulaire_inf (M, E)`, les matrices **M** et **E** deviennent ainsi :

Matrice M triangulaire inférieure	Matrice E
$\begin{bmatrix} 1.25 & 0. & 0. & 0. \\ 3.33 & 2.67 & 0. & 0. \\ 1.67 & -3.67 & -1. & 0. \\ -1. & -2. & 3. & -3. \end{bmatrix}$	$\begin{bmatrix} 1. & -0.12 & 0. & -0.62 \\ 0. & 1. & 0. & -0.33 \\ 0. & 0. & 1. & 0.33 \\ 0. & 0. & 0. & 1. \end{bmatrix}$

III. 7- Transformer la matrice triangulaire inférieure M en matrice diagonale :

Les deux matrices **M** et **E** sont les résultats de la fonction `triangulaire_inf (M, E)`. La matrice **M** est devenue triangulaire inférieure. Écrire la fonction : `diagonale (M, E)` , qui transforme la matrice **M** en matrice diagonale. Les transformations effectuées sur la matrice **M**, doivent être aussi effectuées simultanément sur la matrice **E**.

Exemple :

Après l'appel de la fonction `diagonale (M, E)`, les matrices **M** et **E** deviennent ainsi :

Matrice M diagonale	Matrice E
$\begin{bmatrix} 1.25 & 0. & 0. & 0. \\ 0. & 2.67 & 0. & 0. \\ 0. & 0. & -1. & 0. \\ 0. & 0. & 0. & -3. \end{bmatrix}$	$\begin{bmatrix} 1. & -0.12 & 0. & -0.62 \\ -2.67 & 1.33 & 0. & 1.33 \\ -5. & 2. & 1. & 3. \\ -16.2 & 6.9 & 3. & 10.5 \end{bmatrix}$

III. 8- Calcul de la matrice inverse par élimination de « Gauss-Jordan » :

Écrire la fonction : `inverse (M)` , qui reçoit en paramètre une matrice carrée inversible **M**, et qui calcule et renvoie la matrice inverse de **M**. (Ne pas utiliser la méthode prédéfinie `inv()` de *Python*)

Exemple :

M matrice carrée inversible : La fonction `inverse (M)` renvoie la matrice inverse de **M** :

$\begin{bmatrix} 1. & -1. & 2. & -2. \\ 3. & 2. & 1. & -1. \\ 2. & -3. & -2. & 1. \\ -1. & -2. & 3. & -3. \end{bmatrix}$	$\begin{bmatrix} 0.8 & -0.1 & 0. & -0.5 \\ -1. & 0.5 & 0. & 0.5 \\ 5. & -2. & -1. & -3. \\ 5.4 & -2.3 & -1. & -3.5 \end{bmatrix}$
--	---

III. 9- Matrice stockée dans un fichier texte :

On suppose avoir créé un fichier texte contenant une matrice carrée. Chaque ligne du fichier contient une ligne de la matrice, et les éléments de chaque ligne du fichier sont séparés par le caractère espace.

Exemple :

5.7	3.4	0.0	-0.2	-4.5	-5.0
-5.0	1.0	2.0	5.0	1.0	-0.9
-2.0	1.7	-1.0	-4.0	2.0	2.0
-5.0	5.0	4.0	0.3	-4.0	-1.0
-3.3	1.0	5.0	-4.0	1.0	1.0
-1.0	-2.0	-1.0	5.0	0.1	3.0

Matrice carrée inversible d'ordre 6, stockée dans un fichier texte

Écrire la fonction : `matrice_fichier (ch)` , qui reçoit en paramètre une chaîne de caractère **ch**, qui contient le chemin absolu du fichier texte, contenant une matrice carrée :

- ❖ Si cette matrice est inversible, la fonction renvoie sa matrice inverse ;
- ❖ Si cette matrice n'est pas inversible, la fonction affiche le message « Matrice non inversible » ;
- ❖ Si le fichier texte ne se trouve pas à l'emplacement spécifié dans la chaîne **ch**, l'exception `FileNotFoundException` sera levée. La fonction affiche le message : « Fichier texte inexistant »

----- FIN DE L'ÉPREUVE -----

Partie II : Calcul numériqueInterpolation polynômiale

Dans cette partie, on suppose que les modules **numpy** et **matplotlib.pyplot** sont importés

```
import numpy as np
import matplotlib.pyplot as plt
```

En analyse numérique, l'**interpolation polynômiale** est une technique d'interpolation d'un ensemble de données ou d'une fonction par un polynôme.

Étant donnés n points dans le plan $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$, avec les x_i distincts deux à deux, on cherche le polynôme A de degré au plus $n-1$, et qui passe par ces points.

Supposons que le polynôme d'interpolation est le suivant :

$$A(x) = a_0 + a_1x^1 + a_2x^2 + \dots + a_{n-1}x^{n-1} \quad (1)$$

Afin que le polynôme $A(x)$ passe par l'ensemble des points (x_i, y_i) à interpoler, il doit vérifier : $A(x_i) = y_i$

Ainsi, on obtient un système d'équations linéaires $M^*A=Y$, dont l'écriture matricielle est la suivante :

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-2} & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-2} & x_1^{n-1} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-2} & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ \vdots \\ a_{n-2} \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

Pour construire le polynôme $A(x)$, il faudra donc résoudre ce système afin d'obtenir les valeurs des coefficients a_i du polynôme $A(x)$.

Q.1- Écrire la fonction **découpe (P)** qui reçoit en paramètre une liste **P** de tuples qui représentent les points à interpoler. La fonction construit et retourne deux vecteurs **X** et **Y** qui contiennent respectivement les abscisses et les ordonnées des points de **P**.

Exemple :

$P = [(-1, 2), (0, 1), (5, 4), (1, 3), (-2, 8), (-7, 3), (3, -3)]$

La fonction **découpe (P)** retourne les vecteurs :

$X = [-1, 0, 5, 1, -2, -7, 3]$ et $Y = [2, 1, 4, 3, 8, 3, -3]$

et qui construit et retourne la matrice **M** suivante :

Q.2- Écrire la fonction **matrice (X)** qui reçoit en paramètre la liste **X** des abscisses x_i des points à interpoler,

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-2} & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-2} & x_1^{n-1} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-2} & x_{n-1}^{n-1} \end{pmatrix}$$

Exemple :

$X = [-1, 0, 5, 1, -2, -7, 3]$

La fonction **matrice (X)** retourne la matrice **M** suivante :

$$\begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 5 & 25 & 125 & 625 & 3125 & 156250 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -2 & 4 & -8 & 16 & -32 & 64 \\ 1 & -7 & 49 & -343 & 2401 & -16807 & 117649 \\ 1 & 3 & 9 & 27 & 81 & 243 & 729 \end{bmatrix}$$

Le module **numpy** contient un sous module **linalg**. Ce sous module contient la méthode **solve** qui permet de résoudre le système $M^*A=Y$.

La méthode **solve** reçoit en paramètres la matrice **M** et le vecteur **Y**, et elle retourne le vecteur **A**, solution du système, et qui contient les coefficients du polynôme **A(x)**.

Q.3- Écrire la fonction **polynome (P)** qui reçoit en paramètre la liste **P** de tuples qui représentent les points à interpoler. La fonction calcule et retourne le vecteur **A** qui contient les coefficients du polynôme interpolateur passant par ces points.

Exemple :

$X = [-1, 0, 5, 1, -2, -7, 3]$ et $Y = [2, 1, 4, 3, 8, 3, -3]$

La fonction **polynome (X, Y)** retourne le vecteur **A** suivant :

$[1., 1.18720238, 1.643125, -0.71178571, -0.14696429, 0.02458333, 0.00383929]$

Q.4- Écrire la fonction **image (A, x)** qui reçoit en paramètres un réel **x**, et le vecteur **A** contenant les coefficients du polynôme interpolateur **A(x)**. La fonction retourne l'image de **x** par le polynôme interpolateur **A**.

Exemple :

$A = [1., 1.18720238, 1.643125, -0.71178571, -0.14696429, 0.02458333, 0.00383929]$

La fonction **valeur** (A, d) retourne la valeur suivante :

$$1 \cdot 4^0 + 1.18720238 \cdot 4^1 + 1.643125 \cdot 4^2 + -0.71178571 \cdot 4^3 + \dots + 0.00383929 \cdot 4^6 = -10.23928571428571$$

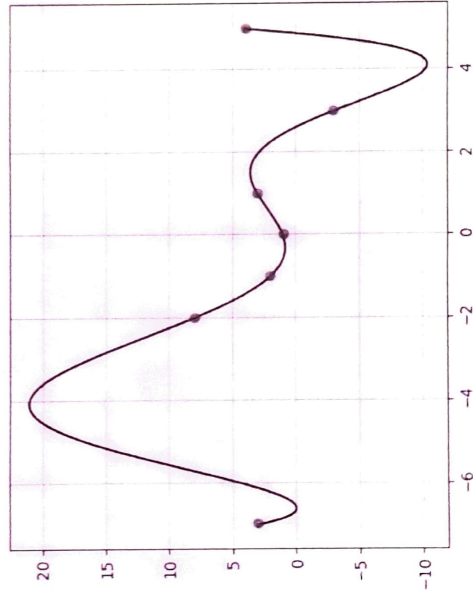
Q.5- Écrire la fonction **courbe** (P) qui reçoit en paramètre la liste P de tuples qui représentent les points à interpoler. La fonction trace la représentation graphique du polynôme interpolateur $A(x)$ qui passe par tous les points de P .

Le nombre de points générés dans la courbe est : **500**

Exemple :

$P = [(-1, 2), (0, 1), (5, 4), (1, 3), (-2, 8), (-7, 3), (3, -3)]$

Après l'appel de la fonction **courbe** (P), on obtient la représentation du polynôme interpolateur qui passe par tous les points de P .



.....
.....
.....